

Does anyone here know Hubble's
Law? (Without googling it or
sneaking a peak at later slides)

Astronomer who made critical contributions to our understanding of the universe. Moved to nearby Wheaton when he was 11, and attended University of Chicago



Edwin Hubble and his famous paper

<https://www.pnas.org/content/pnas/15/3/168.full.pdf>

A RELATION BETWEEN DISTANCE AND RADIAL VELOCITY AMONG EXTRA-GALACTIC NEBULAE

BY EDWIN HUBBLE

MOUNT WILSON OBSERVATORY, CARNEGIE INSTITUTION OF WASHINGTON

Communicated January 17, 1929

https://en.wikipedia.org/wiki/Triangulum_Galaxy



TABLE 1
NEBULAE WHOSE DISTANCES HAVE BEEN ESTIMATED FROM STARS INVOLVED OR FROM
MEAN LUMINOSITIES IN A CLUSTER

OBJECT	m_s	r	v	m_t	M_t
S. Mag.	..	0.032	+ 170	1.5	-16.0
L. Mag.	..	0.034	+ 290	0.5	17.2
N. G. C. 6822	..	0.214	- 130	9.0	12.7
598	..	0.263	- 70	7.0	15.1
221	..	0.275	- 185	8.8	13.4
224	..	0.275	- 220	5.0	17.2
5457	17.0	0.45	+ 200	9.9	13.3
4736	17.3	0.5	+ 290	8.4	15.1
5194	17.3	0.5	+ 270	7.4	16.1
4449	17.8	0.63	+ 200	9.5	14.5
4214	18.3	0.8	+ 300	11.3	13.2
3031	18.5	0.9	- 30	8.3	16.4
3627	18.5	0.9	+ 650	9.1	15.7
4826	18.5	0.9	+ 150	9.0	15.7
5236	18.5	0.9	+ 500	10.4	14.4
1068	18.7	1.0	+ 920	9.1	15.9
5055	19.0	1.1	+ 450	9.6	15.6
7331	19.0	1.1	+ 500	10.4	14.8
4258	19.5	1.4	+ 500	8.7	17.0
4151	20.0	1.7	+ 960	12.0	14.2
4382	..	2.0	+ 500	10.0	16.5
4472	..	2.0	+ 850	8.8	17.7
4486	..	2.0	+ 800	9.7	16.8
4649	..	2.0	+1090	9.5	17.0
Mean					-15.5

m_s = photographic magnitude of brightest stars involved.

r = distance in units of 10^4 parsecs. The first two are Shapley's values.

v = measured velocities in km./sec. N. G. C. 6822, 221, 224 and 5457 are recent determinations by Humason.

m_t = Holetschek's visual magnitude as corrected by Hopmann. The first three objects were not measured by Holetschek, and the values of m_t represent estimates by the author based upon such data as are available.

M_t = total visual absolute magnitude computed from m_t and r .

Triangulum Galaxy (NGC 598)

Edwin Hubble and his famous paper

<https://www.pnas.org/content/pnas/15/3/168.full.pdf>

A RELATION BETWEEN DISTANCE AND RADIAL VELOCITY AMONG EXTRA-GALACTIC NEBULAE

BY EDWIN HUBBLE

MOUNT WILSON OBSERVATORY, CARNEGIE INSTITUTION OF WASHINGTON

Communicated January 17, 1929

1 pc = 3.1×10^{16} m
Positive v implies
away from us,
negative toward us

TABLE 1
NEBULAE WHOSE DISTANCES HAVE BEEN ESTIMATED FROM STARS INVOLVED OR FROM
MEAN LUMINOSITIES IN A CLUSTER

OBJECT	m_s	r	v	m_t	M_t
S. Mag.	..	0.032	+ 170	1.5	-16.0
L. Mag.	..	0.034	+ 290	0.5	17.2
N. G. C. 6822	..	0.214	- 130	9.0	12.7
598	..	0.263	- 70	7.0	15.1
221	..	0.275	- 185	8.8	13.4
224	..	0.275	- 220	5.0	17.2
5457	17.0	0.45	+ 200	9.9	13.3
4736	17.3	0.5	+ 290	8.4	15.1
5194	17.3	0.5	+ 270	7.4	16.1
4449	17.8	0.63	+ 200	9.5	14.5
4214	18.3	0.8	+ 300	11.3	13.2
3031	18.5	0.9	- 30	8.3	16.4
3627	18.5	0.9	+ 650	9.1	15.7
4826	18.5	0.9	+ 150	9.0	15.7
5236	18.5	0.9	+ 500	10.4	14.4
1068	18.7	1.0	+ 920	9.1	15.9
5055	19.0	1.1	+ 450	9.6	15.6
7331	19.0	1.1	+ 500	10.4	14.8
4258	19.5	1.4	+ 500	8.7	17.0
4151	20.0	1.7	+ 960	12.0	14.2
4382	..	2.0	+ 500	10.0	16.5
4472	..	2.0	+ 850	8.8	17.7
4486	..	2.0	+ 800	9.7	16.8
4649	..	2.0	+1090	9.5	17.0
Mean					-15.5

m_s = photographic magnitude of brightest stars involved.

r = distance in units of 10^4 parsecs. The first two are Shapley's values.

v = measured velocities in km./sec. N. G. C. 6822, 221, 224 and 5457 are recent determinations by Humason.

m_t = Holetschek's visual magnitude as corrected by Hopmann. The first three objects were not measured by Holetschek, and the values of m_t represent estimates by the author based upon such data as are available.

M_t = total visual absolute magnitude computed from m_t and r .

Triangulum Galaxy (NGC 598)

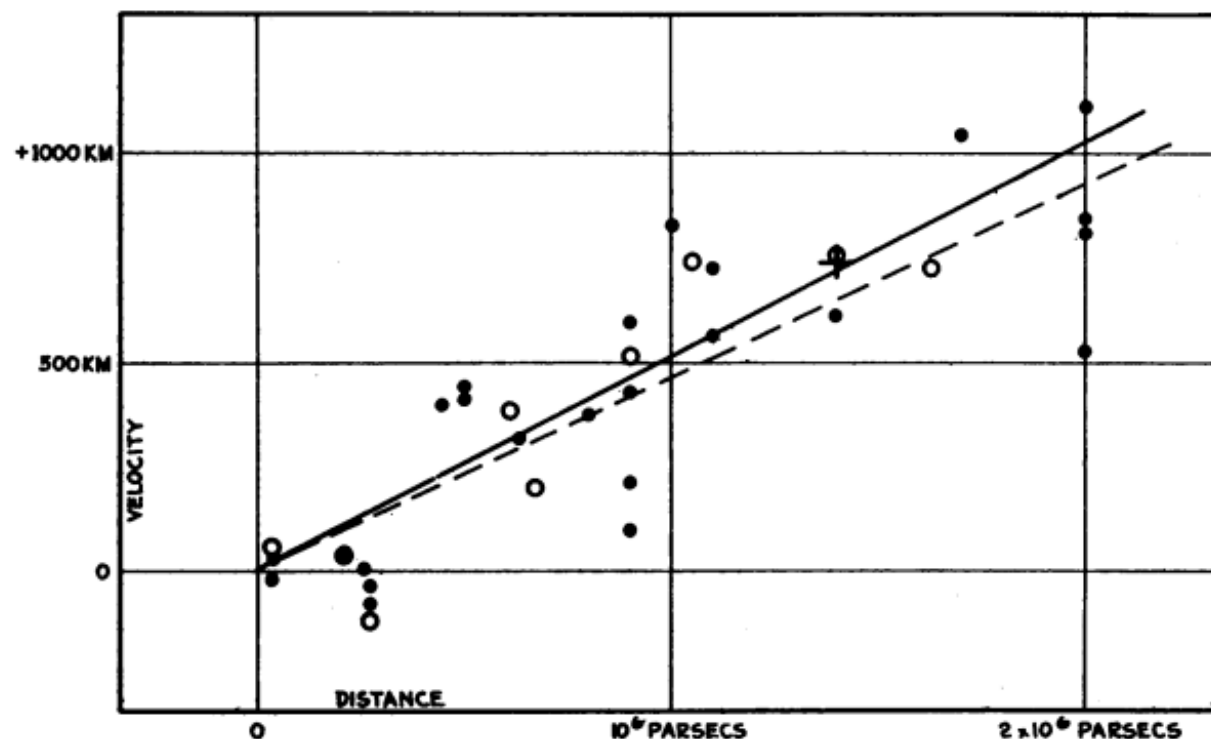


FIGURE 1

Velocity-Distance Relation among Extra-Galactic Nebulae.

Radial velocities, corrected for solar motion, are plotted against distances estimated from involved stars and mean luminosities of nebulae in a cluster. The black discs and full line represent the solution for solar motion using the nebulae individually; the circles and broken line represent the solution combining the nebulae into groups; the cross represents the mean velocity corresponding to the mean distance of 22 nebulae whose distances could not be estimated individually.

Clear linear relationship! How does one fit such curves, though?

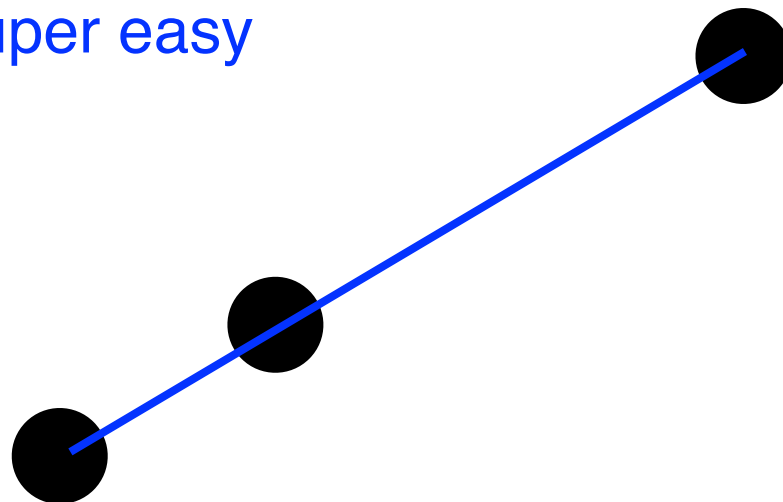
Well, that doesn't help, we can't fit
a straight line to 0 or 1 point!



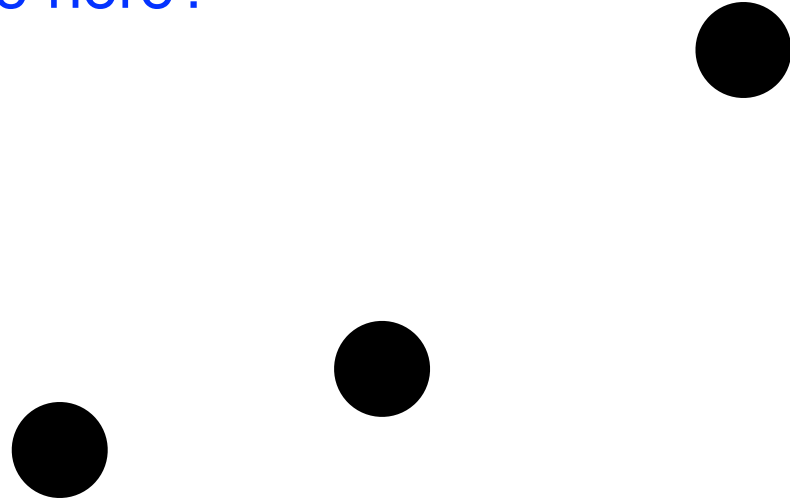
Well, with 2 points, this is easy



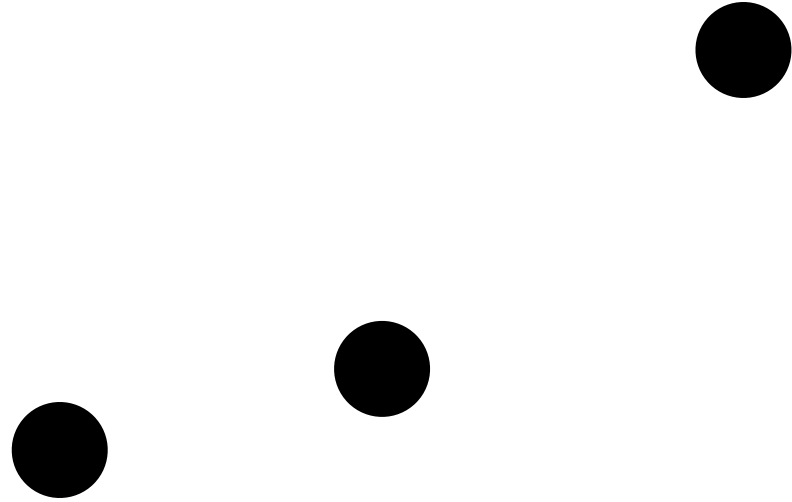
IF the 3 points line up and are colinear, this is also super easy



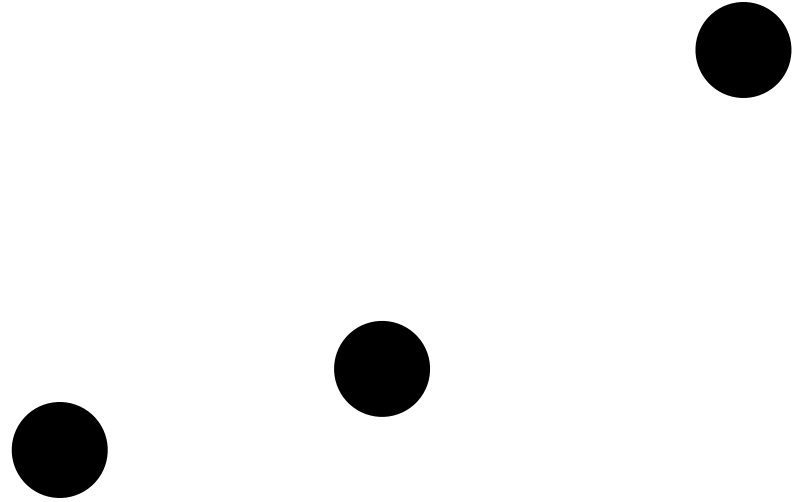
Uh oh, what do we do here?



Well, there are a few reasons why
our points might not all be
colinear

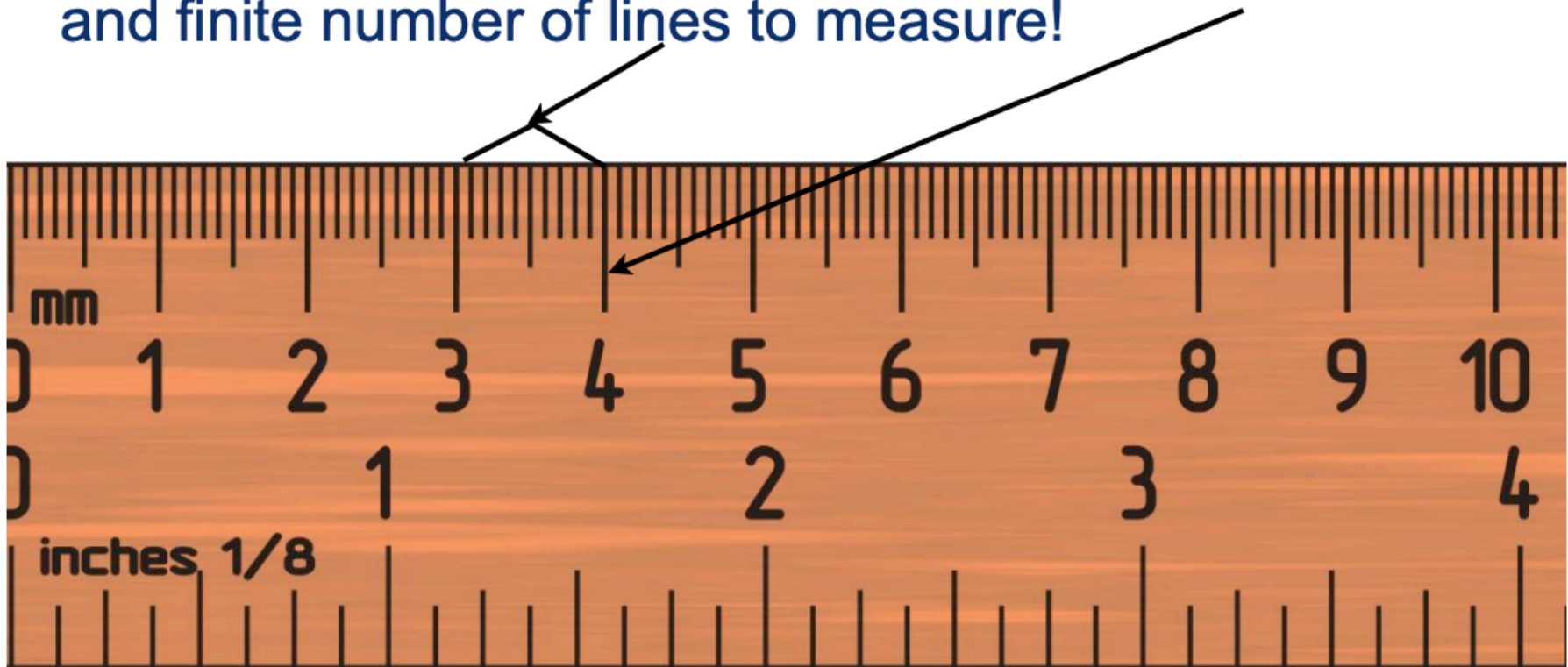


Maybe each point is the result of statistical sampling. We repeat some measurement/sampling over and over; if so, this is easy to estimate and understand



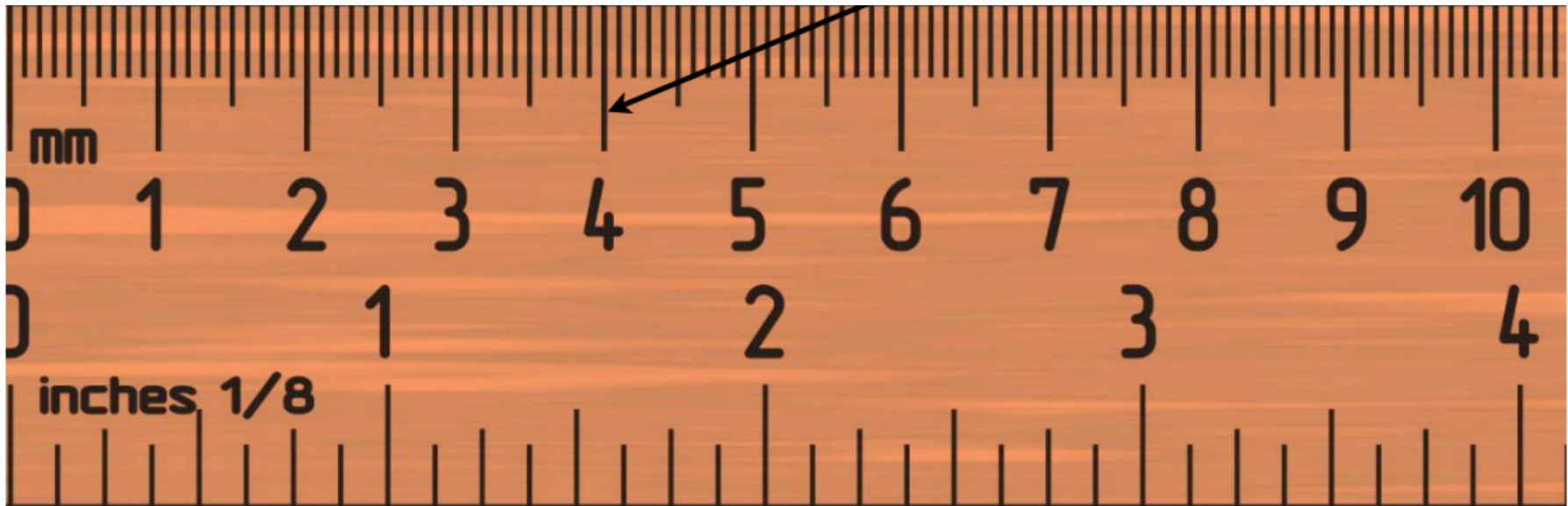
We can also have systematic uncertainties (maybe our ruler is not correct, maybe our stop watch is wrong, maybe there is human error, or otherwise). This is trickier.

- Example : measuring distance with a ruler
 - Systematic limitation : ruler has finite width of the lines, and finite number of lines to measure!



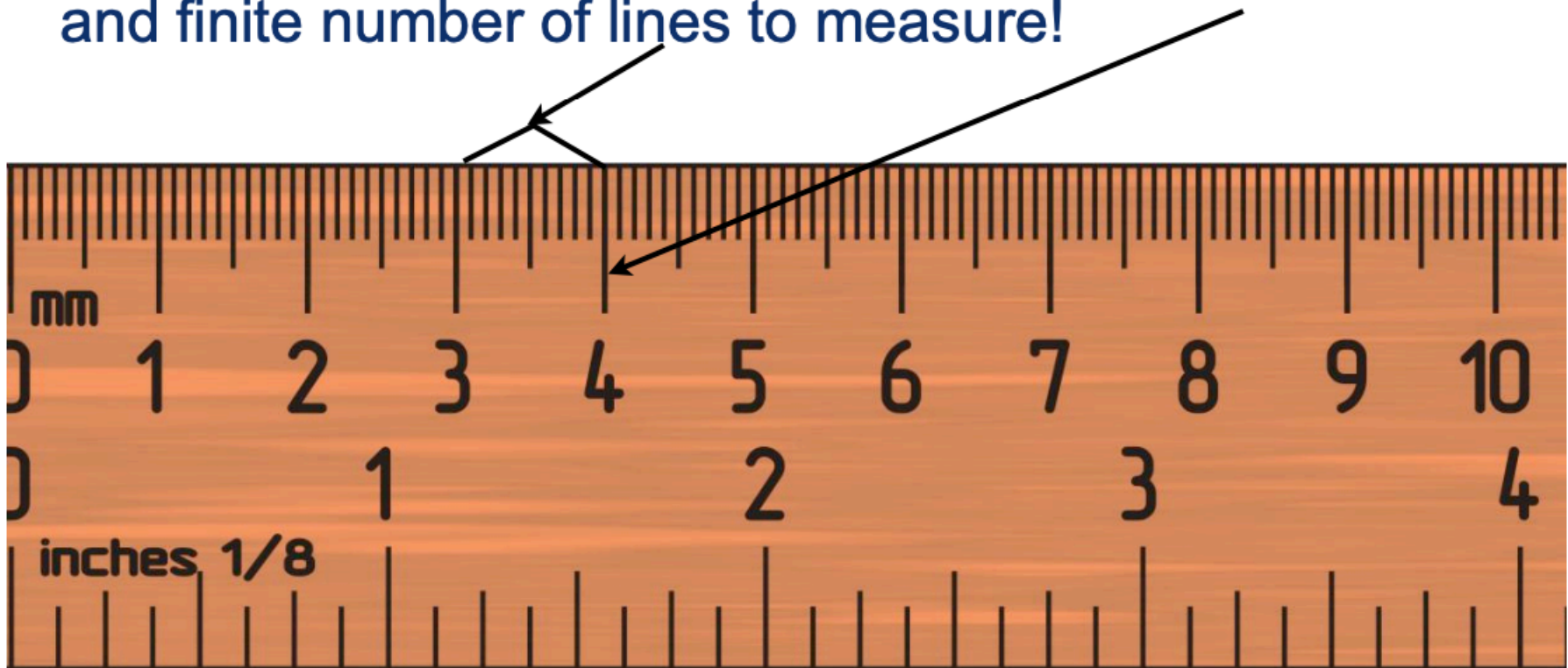
- Statistical variation : you can try to repeat the same measurement over and over to get a better estimate of the “true” value

Repeat more measurements and this
uncertainty shrinks!

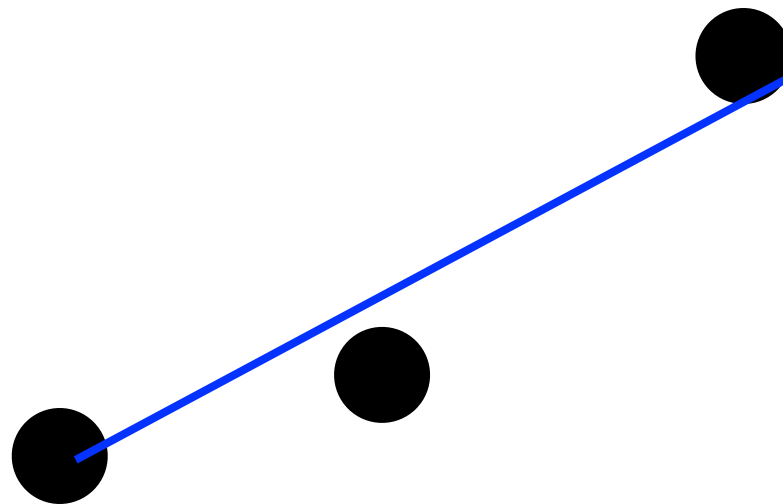


- Statistical variation : you can try to repeat the same measurement over and over to get a better estimate of the “true” value

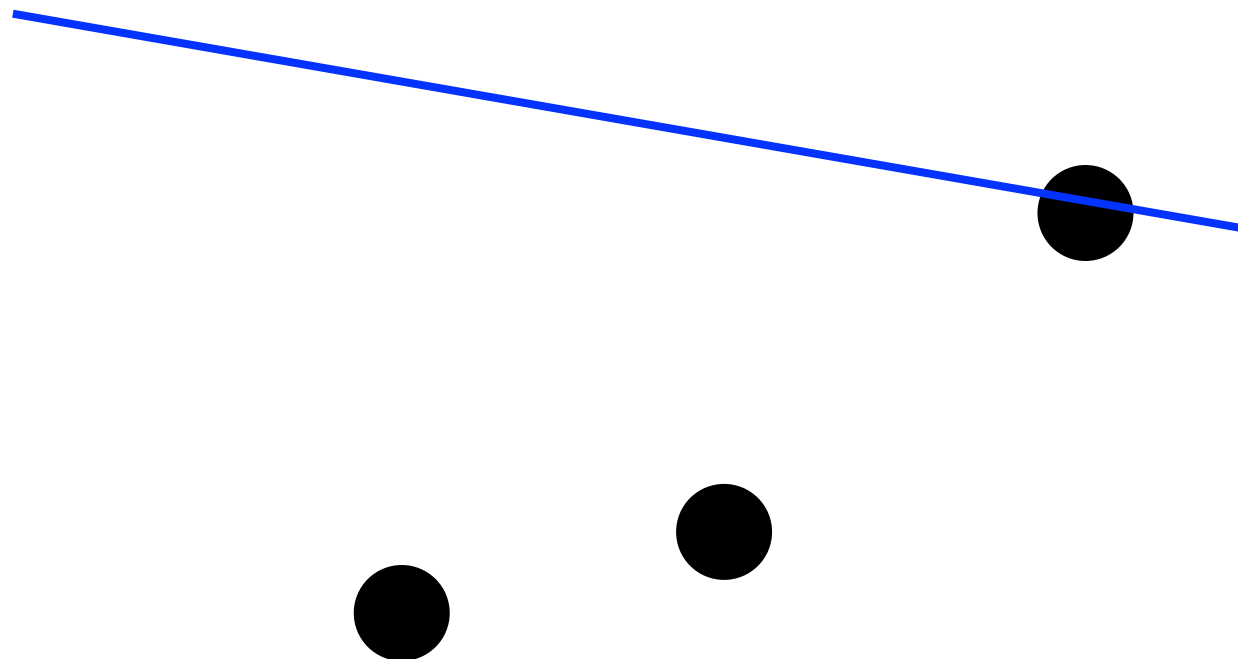
- Example : measuring distance with a ruler
 - Systematic limitation : ruler has finite width of the lines, and finite number of lines to measure!



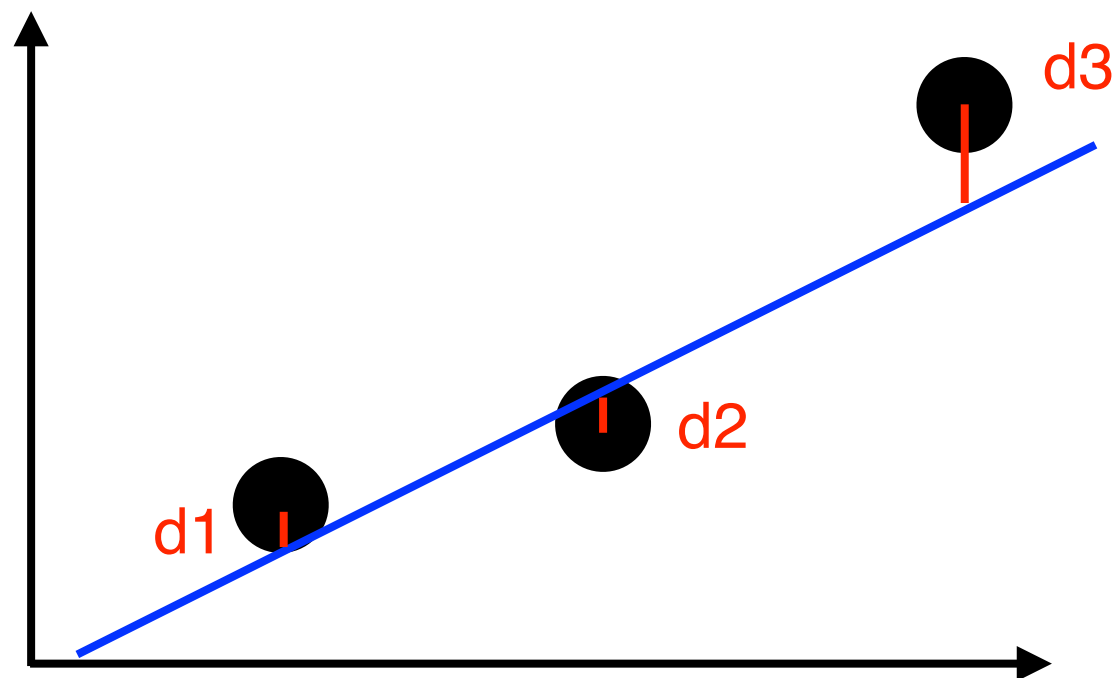
Unfortunately, more trials doesn't help with this. And it's harder to estimate!



This looks like a reasonable guess and a good fit (of course, we will be more mathematically rigorous shortly)

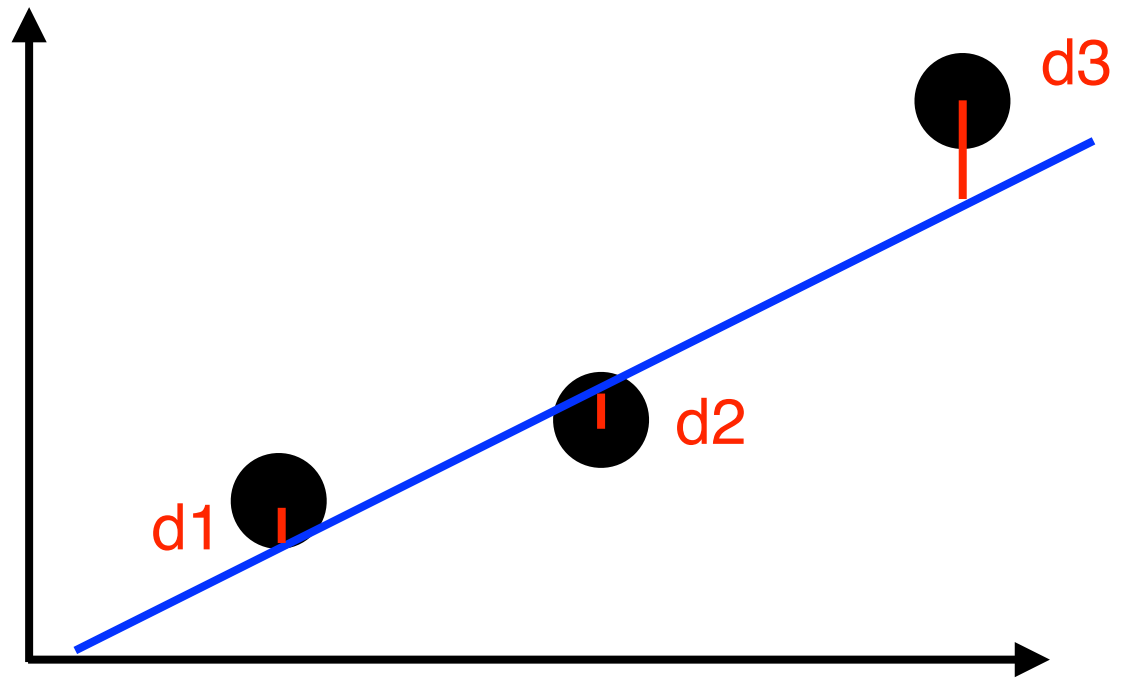


This is clearly not a good fit!



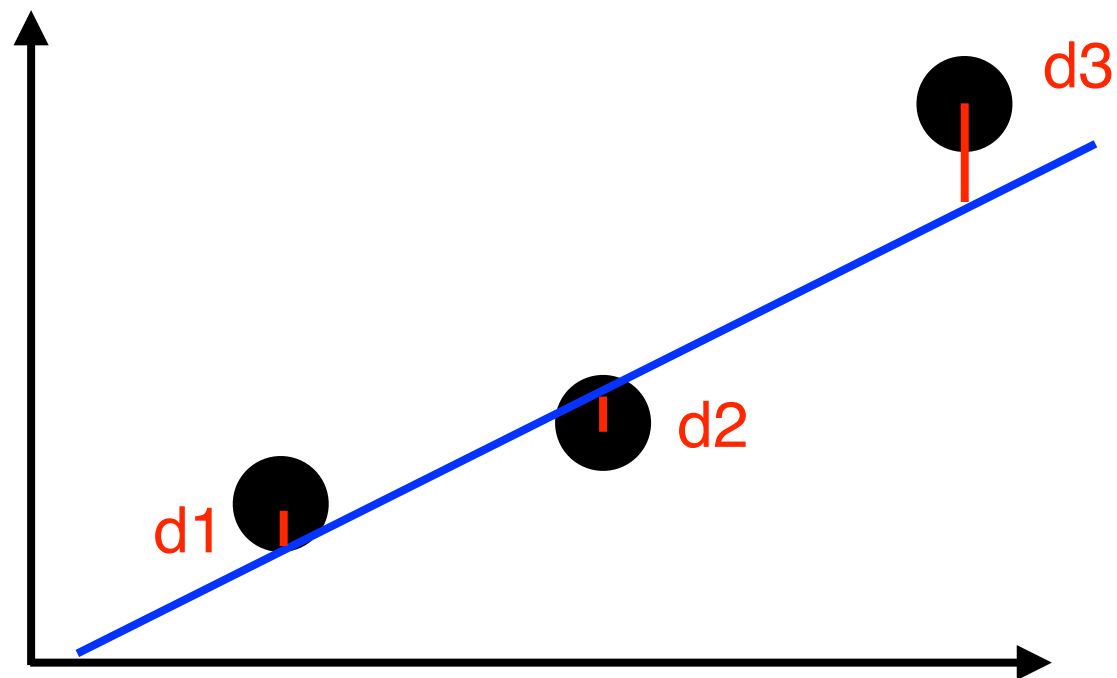
Assume the errors are all the same (this assumption will be relaxed next). Our straight line is given by $y(x) = a + bx$, so that our prediction for point x_i is $y_{\text{pred},i} = a + bx_i$ and then the distance between this and the measured value y_i is given by $y_i - y_{\text{pred},i} = y_i - a - bx_i$

Why does the first line/fit look better?



The distance between the observed and measured value for point i is $y_i - a - bx_i$. Our total “distance” squared is then:

$$f(a, b) = \sum_i (y_i - a - bx_i)^2$$



We want to **minimize** the following quantity:

$$f(a, b) = \sum_i (y_i - a - bx_i)^2$$

We want to **minimize** the following quantity:

$$f(a, b) = \sum_i (y_i - a - bx_i)^2$$

$$\frac{\partial f}{\partial a} = -2 \sum_i (y_i - a - bx_i)$$

$$\frac{\partial f}{\partial b} = -2 \sum_i x_i (y_i - a - bx_i)$$

Two equations,
two unknowns
(a,b)

$$\frac{\partial f}{\partial a} = -2 \sum_i (y_i - a - bx_i) = 0$$

$$\frac{\partial f}{\partial b} = -2 \sum_i x_i (y_i - a - bx_i) = 0$$

$$\sum_i (y_i - a - bx_i) = 0$$

$$\sum_i x_i (y_i - a - bx_i) = 0$$

$$\sum_i y_i - Na - b \sum_i x_i = 0$$

$$\sum_i x_i y_i - a \sum_i x_i - b \sum_i x_i^2 = 0$$

$$\sum_i y_i - Na - b \sum x_i = 0$$

$$\sum_i x_i y_i - a \sum x_i - b \sum x_i^2 = 0$$

$$a = \frac{1}{N} \sum y_i - b \frac{1}{N} \sum x_i$$

$$a = \frac{1}{\sum x_i} \sum x_i y_i - b \frac{1}{\sum x_i} \sum x_i^2$$

$$a = \frac{1}{N} \sum y_i - b \frac{1}{N} \sum x_i$$

$$a = \frac{1}{\sum x_i} \sum x_i y_i - b \frac{1}{\sum x_i} \sum x_i^2$$

$$s_y/N - (b/N)s_x = (s_{xy}/s_x) - b(s_{xx}/s_x)$$

$$b(s_{xx}/s_x - s_x/N) = s_{xy}/s_x - s_y/N$$

$$b(Ns_{xx} - s_x^2) = Ns_{xy} - s_x s_y$$

$$b = (Ns_{xy} - s_x s_y) / (Ns_{xx} - s_x * s_x)$$

$$a = \frac{1}{N} \sum y_i - b \frac{1}{N} \sum x_i$$

$$a = \frac{1}{\sum x_i} \sum x_i y_i - b \frac{1}{\sum x_i} \sum x_i^2$$

$$s_y/N - a = (b/N)s_x$$

$$s_{xy}/s_x - a = (b/s_x)s_{xx}$$

$$b = s_y/s_x - aN/s_x = s_{xy}/s_{xx} - as_x/s_{xx}$$

$$s_y/s_x - s_{xy}/s_{xx} = a(N/s_x - s_x/s_{xx})$$

$$a = (s_y/s_x - s_{xy}/s_{xx})/(N/s_x - s_x/s_{xx})$$

$$a = (s_y * s_{xx} - s_{xy} * s_x)/(N * s_{xx} - s_x^2)$$

$$a = (s_y * s_{xx} - s_{xy} * s_x) / (N * s_{xx} - s_x^2)$$

$$b = (N * s_{xy} - s_x * s_y) / (N * s_{xx} - s_x^2)$$

Just a few simple sums to
calculate, pretty straightforward!

Is that the full picture?

We also want to know three additional, very important quantities:

- 1) The uncertainty on a
- 2) The uncertainty on b
- 3) The uncertainty per degree of freedom of the fit (aka the goodness of the fit). For our linear fit we have two constraints so the variance of the fit, or goodness of fit, is:

$$\sigma^2 = \frac{1}{n - 2} \sum (y_i - y(x_i))^2$$

And what do we do if the uncertainty on each point is not equal? Minimize the chi2 instead!

$$\chi^2(a, b) = \sum \left(\frac{y_i - a - bx_i}{\sigma_i} \right)^2$$

And what do we do if the uncertainty on each point is not equal? Minimize the chi2 instead!

$$\chi^2(a, b) = \sum \left(\frac{y_i - a - bx_i}{\sigma_i} \right)^2$$

If the uncertainties on each point are constant, then minimizing the above gives our previous result. If not, it provides more weight in the fit to points with smaller error, and less weight in the fit to points with larger error

Using point-by-point errors

Uncertainties on parameters!

$$b = \frac{1}{S_{tt}} \sum_{i=0}^{n-1} \frac{t_i y_i}{\sigma_i}, \quad a = \frac{S_y - S_x b}{S} \quad \sigma_a^2 = \frac{1}{S} \left(1 + \frac{S_x^2}{S S_{tt}} \right), \quad \sigma_b^2 = \frac{1}{S_{tt}}$$

with

$$t_i = \frac{1}{\sigma_i} \left(x_i - \frac{S_x}{S} \right), \quad S_{tt} = \sum_{i=0}^{n-1} t_i^2,$$

$$S = \sum_{i=0}^{n-1} \frac{1}{\sigma_i^2}, \quad S_x = \sum_{i=0}^{n-1} \frac{x_i}{\sigma_i^2}, \quad S_y = \sum_{i=0}^{n-1} \frac{y_i}{\sigma_i^2}$$

And goodness of fit :

$$\chi^2(a, b) / ndof = \frac{1}{n - 2} \sum \left(\frac{y_i - a - b x_i}{\sigma_i} \right)^2$$

Using point-by-point errors

And goodness of fit :

$$\chi^2(a, b)/ndof = \frac{1}{n - 2} \sum \left(\frac{y_i - a - bx_i}{\sigma_i} \right)^2$$

If our data really follow a linear distribution AND we have estimated our errors correctly, we should get χ^2/ndf values ~ 1 . Either way, we can use the χ^2 and n to calculate a p-value indicating our goodness of fit

How do we fit other curves to our data? One obvious thing to do is to transform the data so that it is a straight line! For example:

$$y = Ae^{Bx} \rightarrow \ln y = \ln A + Bx$$

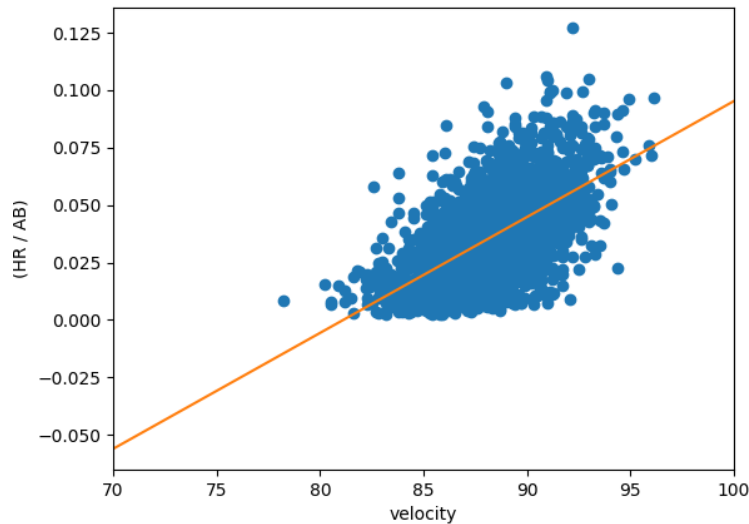
So if we fit $\ln(y)$ as a function of x , we can use the same tools as before, just remembering that B is the slope and our constant is $\ln(A)$

$$y = Ae^{Bx} \rightarrow \ln y = \ln A + Bx$$

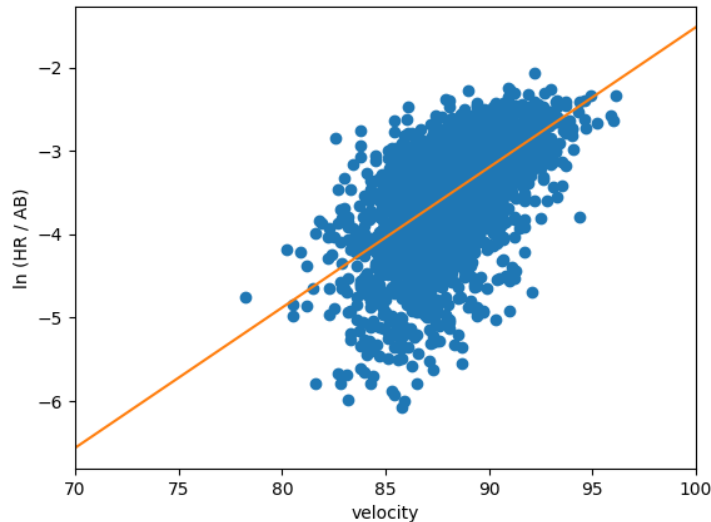
$$y' = f(y) \rightarrow \sigma_{y'}^2 = \left(\frac{\partial f}{\partial y} \right)^2 \sigma_y^2$$

$$y' = \ln y = \ln A + Bx \rightarrow \sigma_{y'} = \frac{\sigma_y}{|y|}$$

Example from baseball

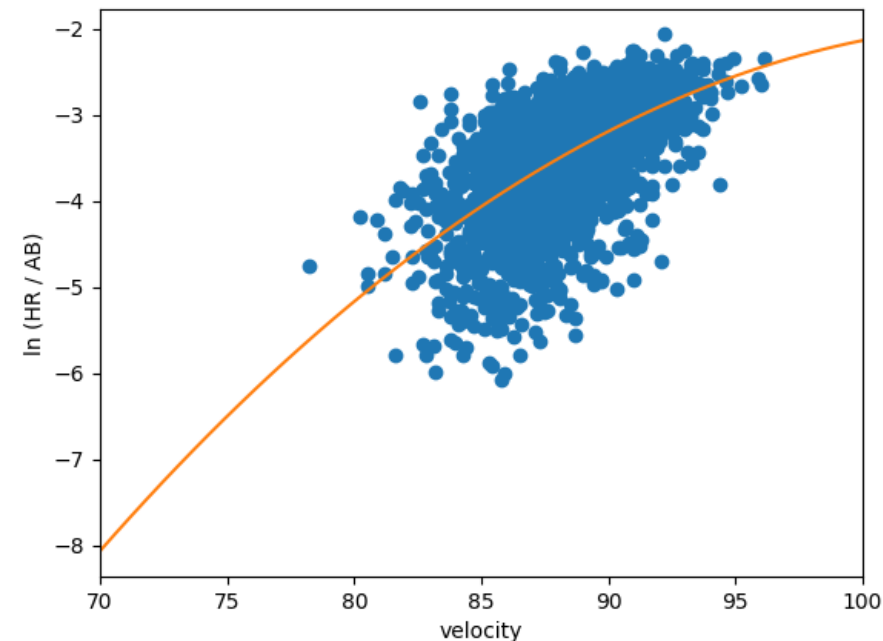


The harder you hit the ball, the more home runs you should hit.
Is a linear fit really right?
Exponential may be better
(need to do goodness of fit checks to decide this!)



Also, quick Q: What is wrong with my plots? (Good check for you to make...)

Example from baseball



Try fitting a 2nd order polynomial to the log
(equivalent to $y = Ae^{Bx + Cx^2}$)

Aside from goodness of fit tests, how might you determine if this is a better fit?

How to fit a more general function?

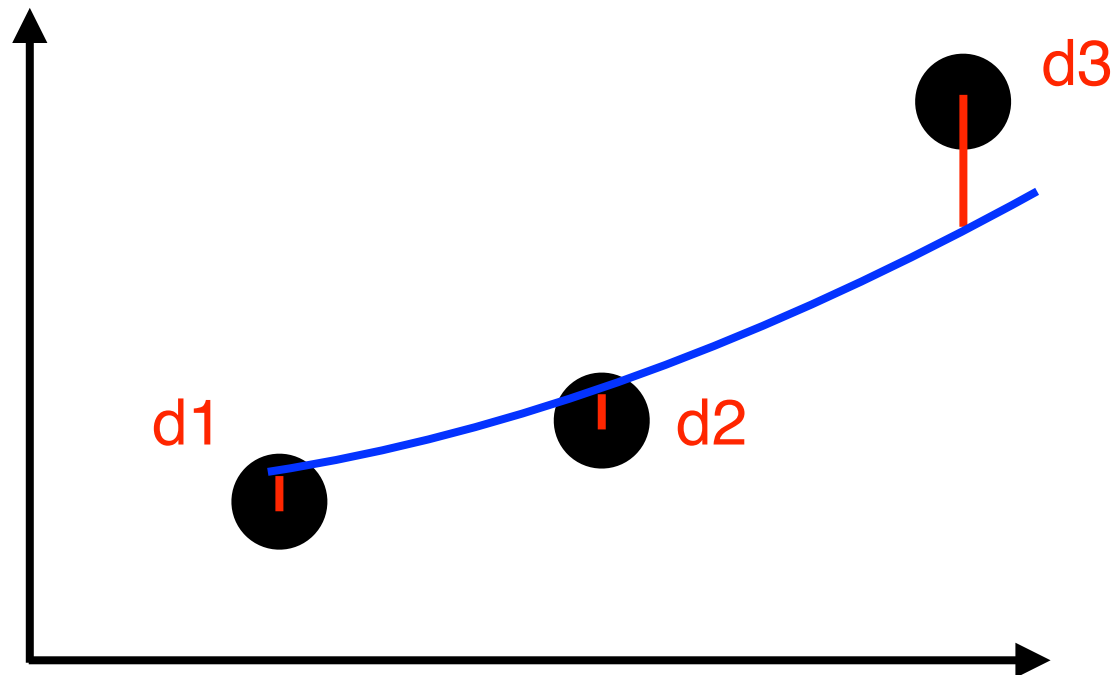
$$y = y(x; \vec{p})$$

y is a function of x , with vector
of parameters $\mathbf{p}$

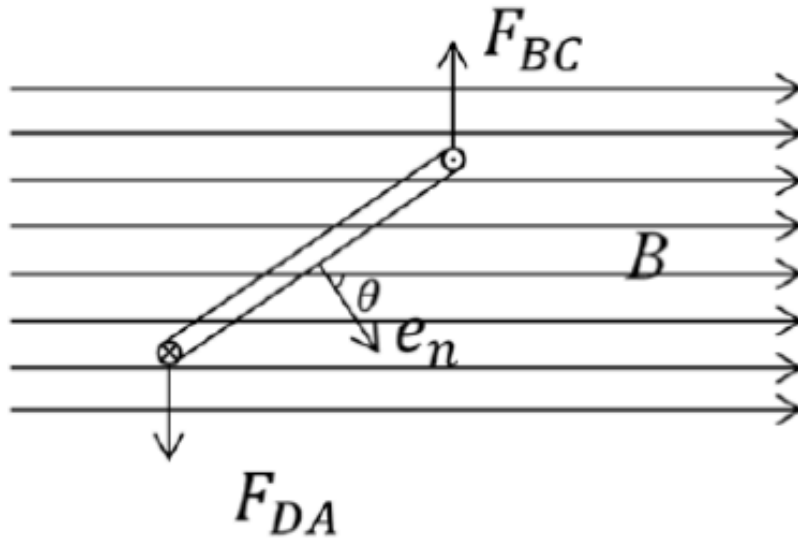
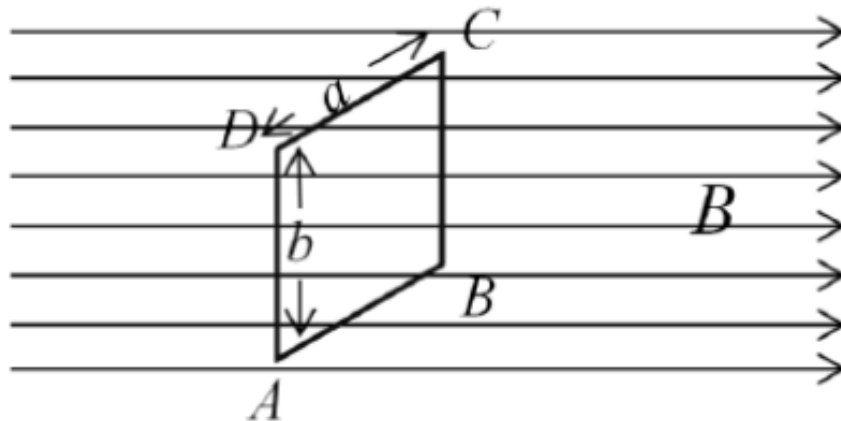
$$\chi^2(\vec{p}) = \sum_i \left(\frac{y_i - y(x; \vec{p})}{\sigma_i} \right)^2$$

Minimize this for $\mathbf{p}$

Still minimizing the
total distance
between prediction
and observation



For the CS folks here... it's OK, don't panic :)



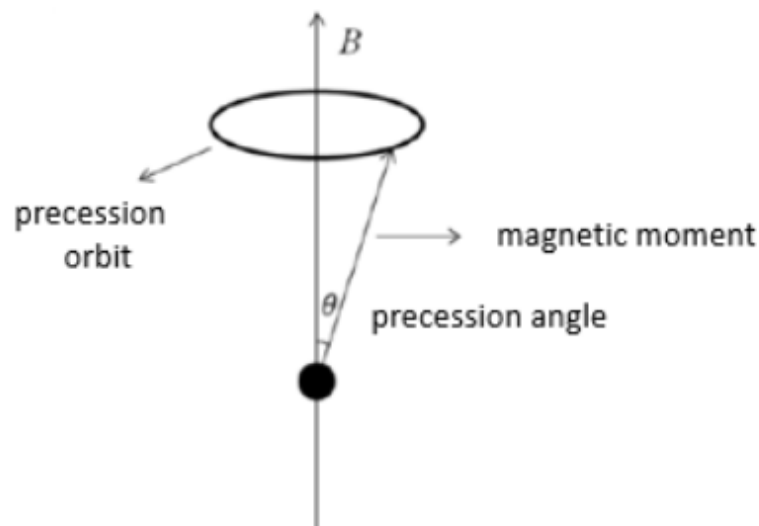
Current I running through the loop. Biot-Savart law tells us that there is a force F on each of the b length arms, with magnitude bIB , and these are in opposite direction so there is a torque τ on the coil. Perpendicular distance between the two forces is $a \sin \theta$, so

$$\tau = abIB \sin \theta = AIB \sin \theta \quad (A=ab)$$

$$|\tau| = IA \vec{e}_n \times \vec{B} = \vec{M} \times \vec{B}$$

$$\vec{M} = IA \vec{e}_n \text{ with } \vec{e}_n \text{ the unit vector normal to the coil}$$

How do we measure this in particle physics?

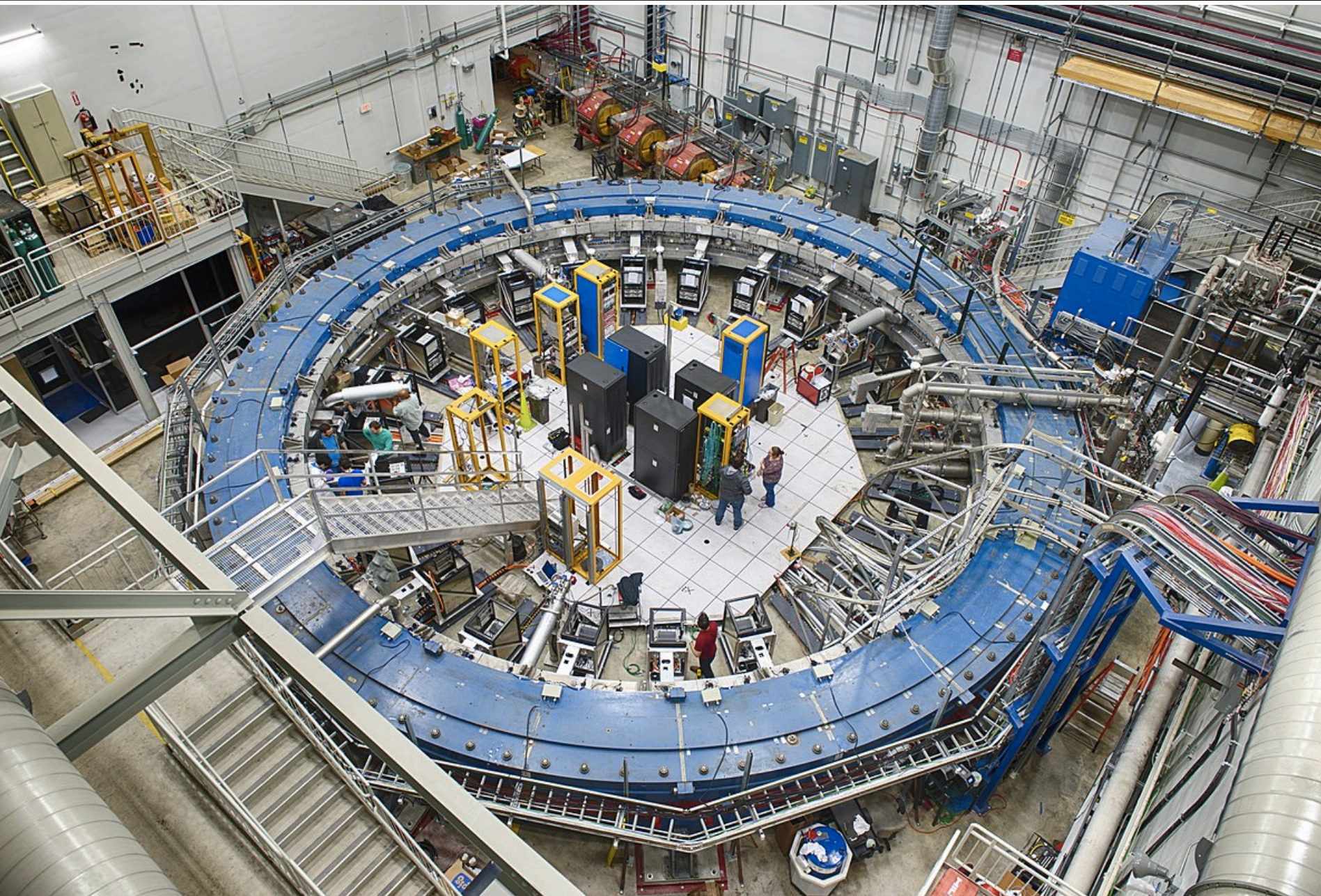


Analogous to a spinning top - spins rotates about the magnetic field

$$\frac{d\vec{S}}{dt} = \vec{L} = \vec{M} \times \vec{B} = g \frac{e}{2m} \vec{S} \times \vec{B}.$$

Challenge - we are in the lab frame, not the rest frame of the muon!

$$\omega = g \frac{e}{2m} B.$$



$$\frac{d\vec{S}}{dt} = (\vec{\omega}_c + \vec{\omega}_a) \times \vec{S},$$

$$\vec{\omega}_c = -\frac{e}{\gamma m} \left(\vec{B} + \frac{\gamma^2}{\gamma^2 - 1} \frac{\vec{E} \times \vec{v}}{c^2} \right),$$

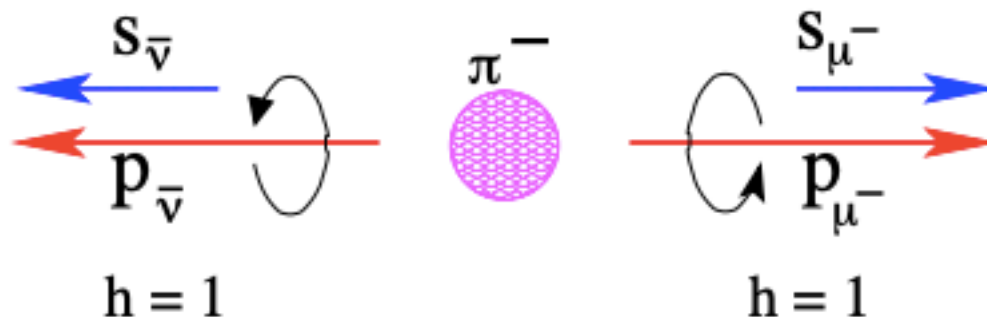
$$\vec{\omega}_a = -\frac{e}{m} \left[a\vec{B} - a \left(\frac{\gamma}{\gamma + 1} \frac{\vec{v} \cdot \vec{B}}{c^2} \right) \vec{v} + \left(a - \frac{1}{\gamma^2 - 1} \right) \frac{\vec{E} \times \vec{v}}{c^2} \right]$$

Pick the velocity giving “magic γ factor” to simplify ω_a . The other challenge is precisely measuring the **B** field and ensuring that it is uniform. Not easy! Use careful NMR probes and other tools

Muons come from pion decays. Thanks to parity violation, these are fully polarized!

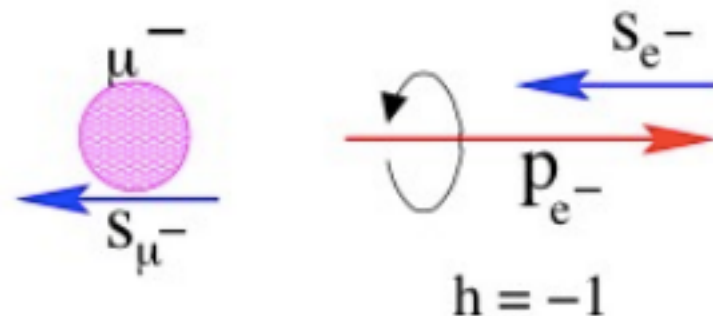
Lucky break from parity violation

https://indico.cern.ch/event/276476/contributions/1620139/attachments/501953/693162/g_minus_2_fewbuildins.pdf

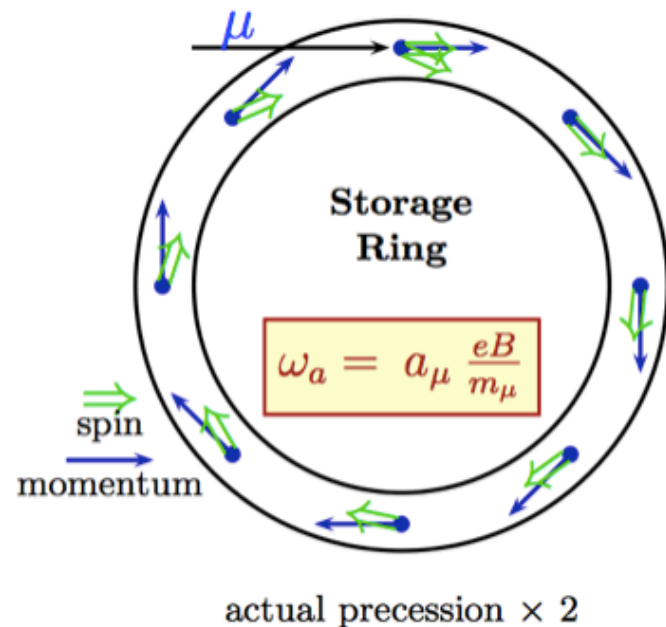


Weak decay correlates muon spin and electron momentum

And there is a correlation between muon's spin and electron momentum from its decay



Spin precesses because $g_\mu \neq 2$



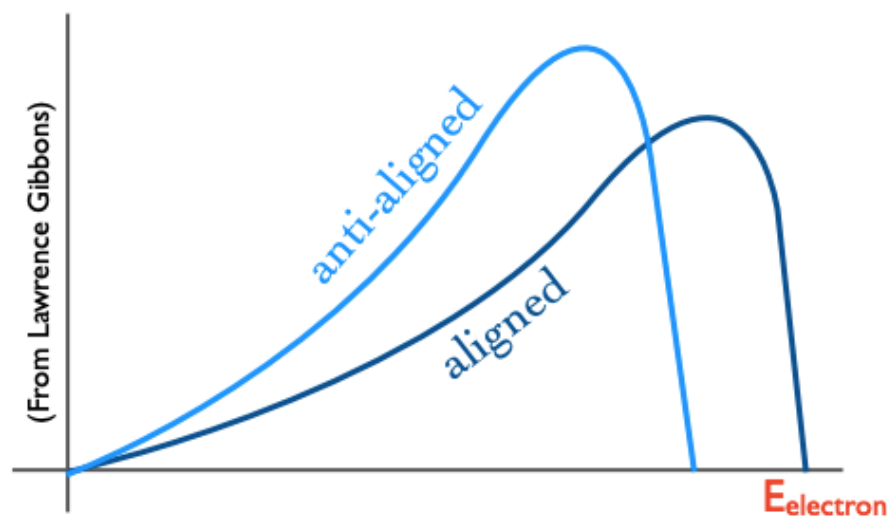
https://indico.cern.ch/event/276476/contributions/1620139/attachments/501953/693162/g_minus_2_fewbuildins.pdf

3. Measure Muon Spin Precession

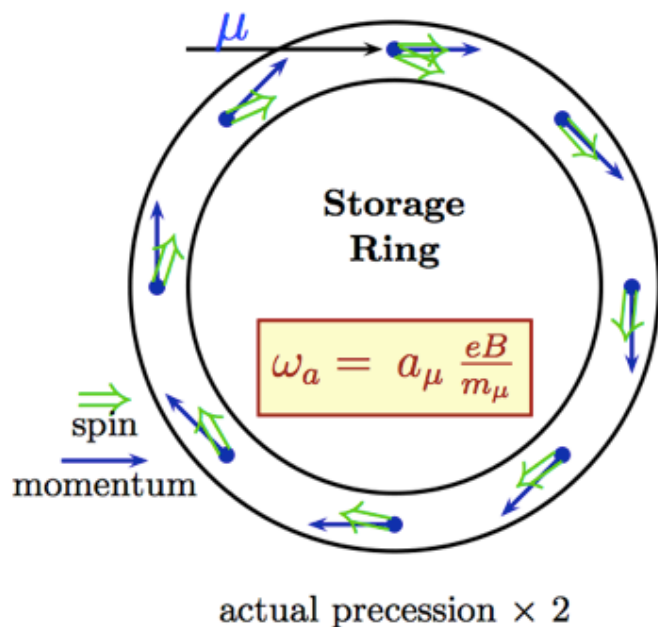


3. Measure Electron Energy

Harder electron spectrum when spin and momentum are aligned



Spin precesses because $g_\mu \neq 2$

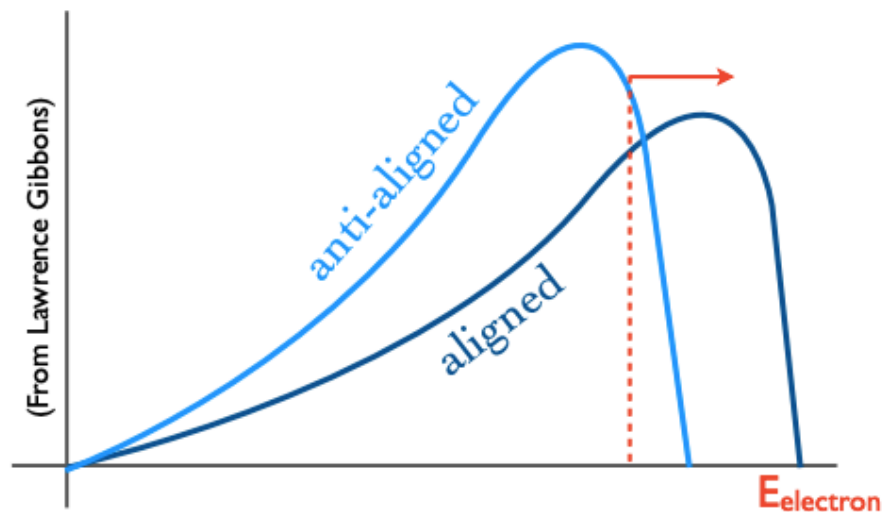


3. Measure Muon Spin Precession



3. Measure Electron Energy

Harder electron spectrum when spin and momentum are aligned



Count $N(e)$ above fixed threshold.
Oscillation rate $\propto a_\mu$

https://indico.cern.ch/event/276476/contributions/1620139/attachments/501953/693162/g_minus_2_fewbuildins.pdf

Not an easy thing to calculate at higher orders!

1805.01944

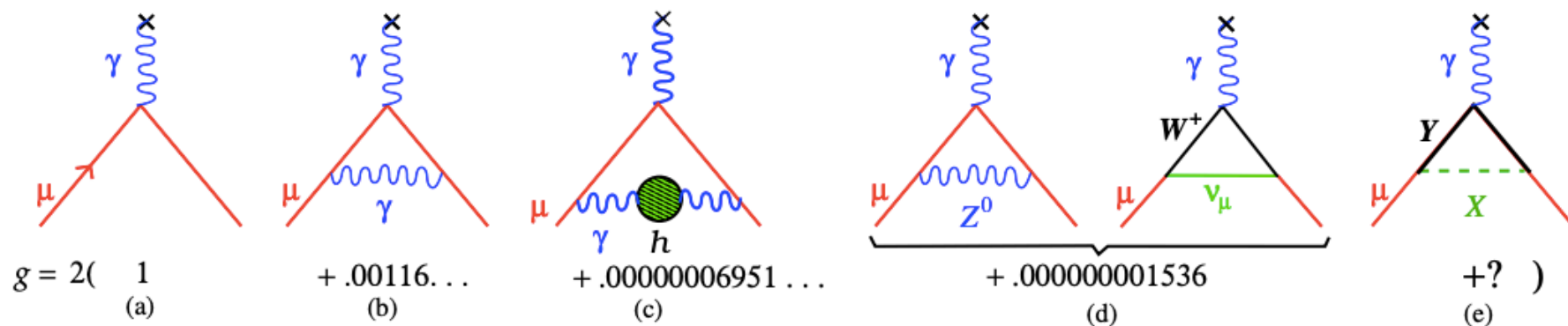


Figure 1: The Feynman graphs showing contributions to g for each of the Standard Model forces, ordered by size: (a) The Dirac interaction. (b) The lowest-order QED term $\alpha/2\pi$, which dominates the value of the anomaly. (c) The hadronic vacuum polarization contribution. (d) The lowest-order electroweak contributions. (The one-loop Higgs contribution is negligible.) (e) Potential contribution from new BSM particles X and Y .

Big challenge for evaluating the potential for BSM physics!
Measurement already accurate to 0.14 ppm!

3.2 The results of the measurements

The BNL E821 result is [2]

$$a_{\mu}^{\text{BNL}} = 116592091(63) \times 10^{-11}. \quad (3.9)$$

The Fermilab result combined with the BNL result is [1]

$$a_{\mu}^{\text{Exp}} = a_{\mu}^{\text{BNL+FNAL}} = 116592061(41) \times 10^{-11}. \quad (3.10)$$

The SM prediction

$$a_{\mu}^{\text{SM}} = 116591810(43) \times 10^{-11}, \quad (3.11)$$

consists of the following contributions [3–6]

$$a_{\mu}^{\text{QED}} = 116584718.9(1) \times 10^{-11}, \text{ 5 loops!} \quad (3.12)$$

$$a_{\mu}^{\text{EW}} = 153.6(1) \times 10^{-11}, \quad (3.13)$$

$$a_{\mu}^{\text{HVP, LO}} = 6931(40) \times 10^{-11}, \quad (3.14)$$

$$a_{\mu}^{\text{HVP, NLO}} = -98.3(7) \times 10^{-11}, \quad (3.15)$$

$$a_{\mu}^{\text{HVP, NNLO}} = 12.4(1) \times 10^{-11}, \quad (3.16)$$

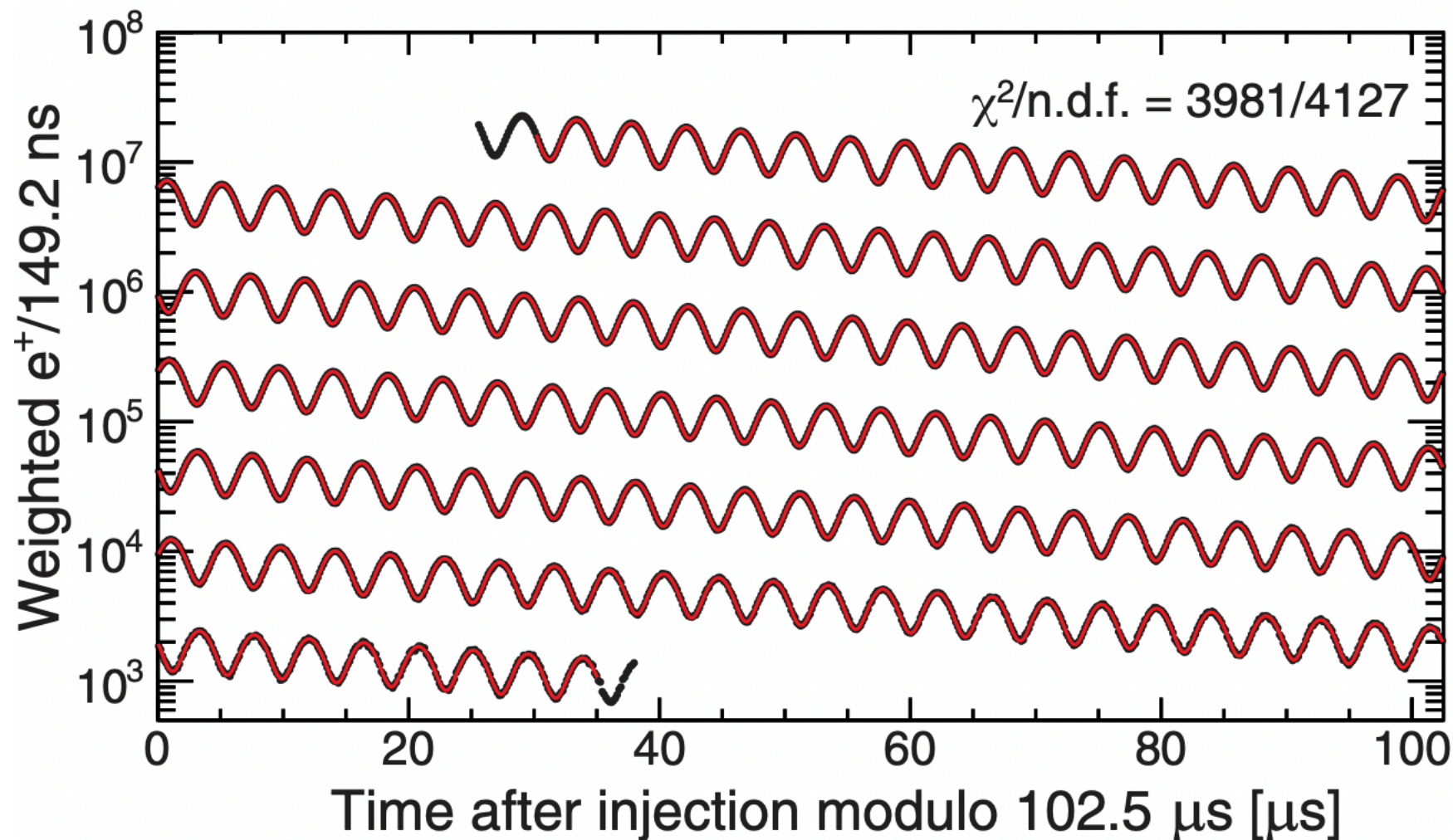
$$a_{\mu}^{\text{HLBL}} + a_{\mu}^{\text{HLBL, NLO}} = 92(18) \times 10^{-11}, \quad (3.17)$$

where the main uncertainties come from the hadronic contributions in HVP and HLBL. The deviation between the experiment and SM is

$$4.2\sigma ! \quad a_{\mu}^{\text{Exp}} - a_{\mu}^{\text{SM}} = 251(59) \times 10^{-11}, \quad (3.18)$$

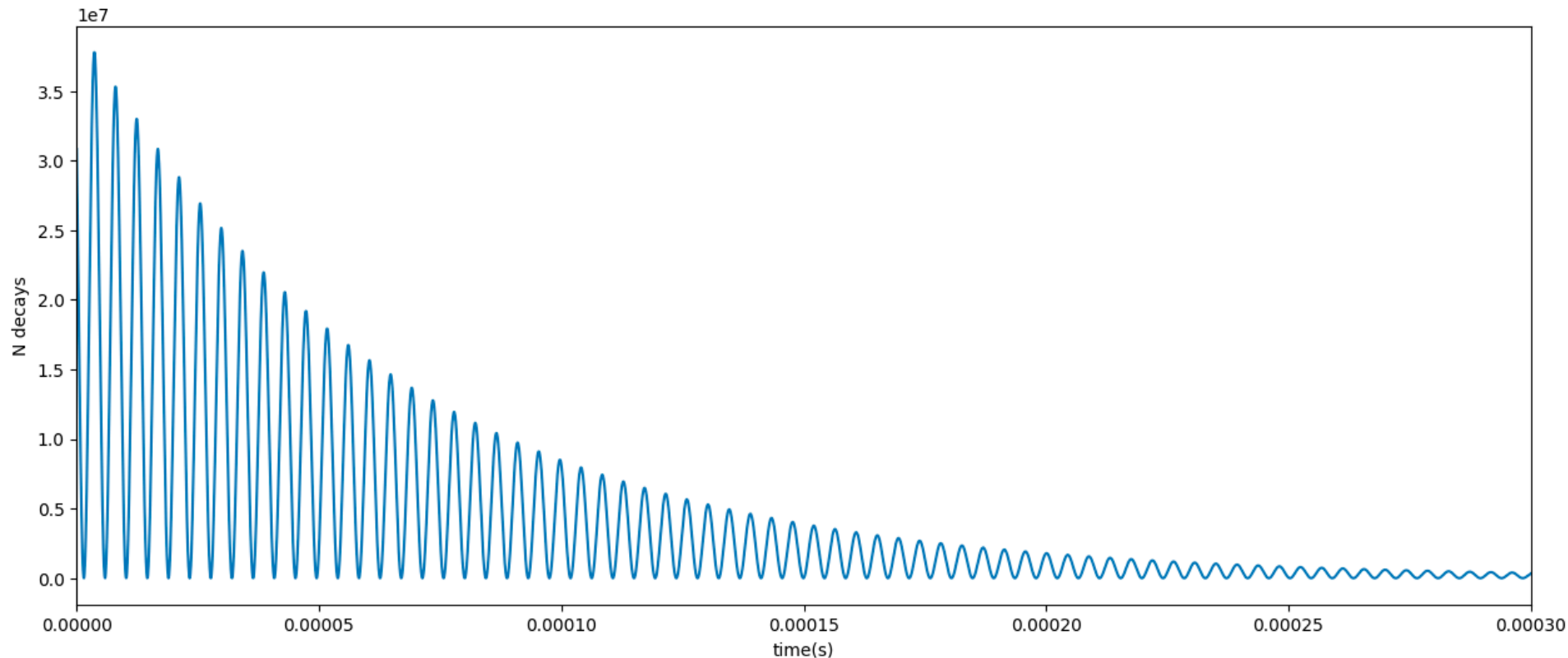
$$N(t) = N_0 \eta_N(t) e^{-t/\tau_\mu} \times \{1 + A \eta_A(t) \cos[\omega_a^m t + \varphi_0 + \eta_\phi(t)]\}$$

The number of measured decays oscillates as a cos term, but with an exponential term as muons decay!



I generated data with a simplified fit model (the actual analysis has a lot of corrections to these terms that we won't account for here!)

$$N(t) = N_0 e^{-t/\tau} (1 + \cos(\omega t + \delta))$$

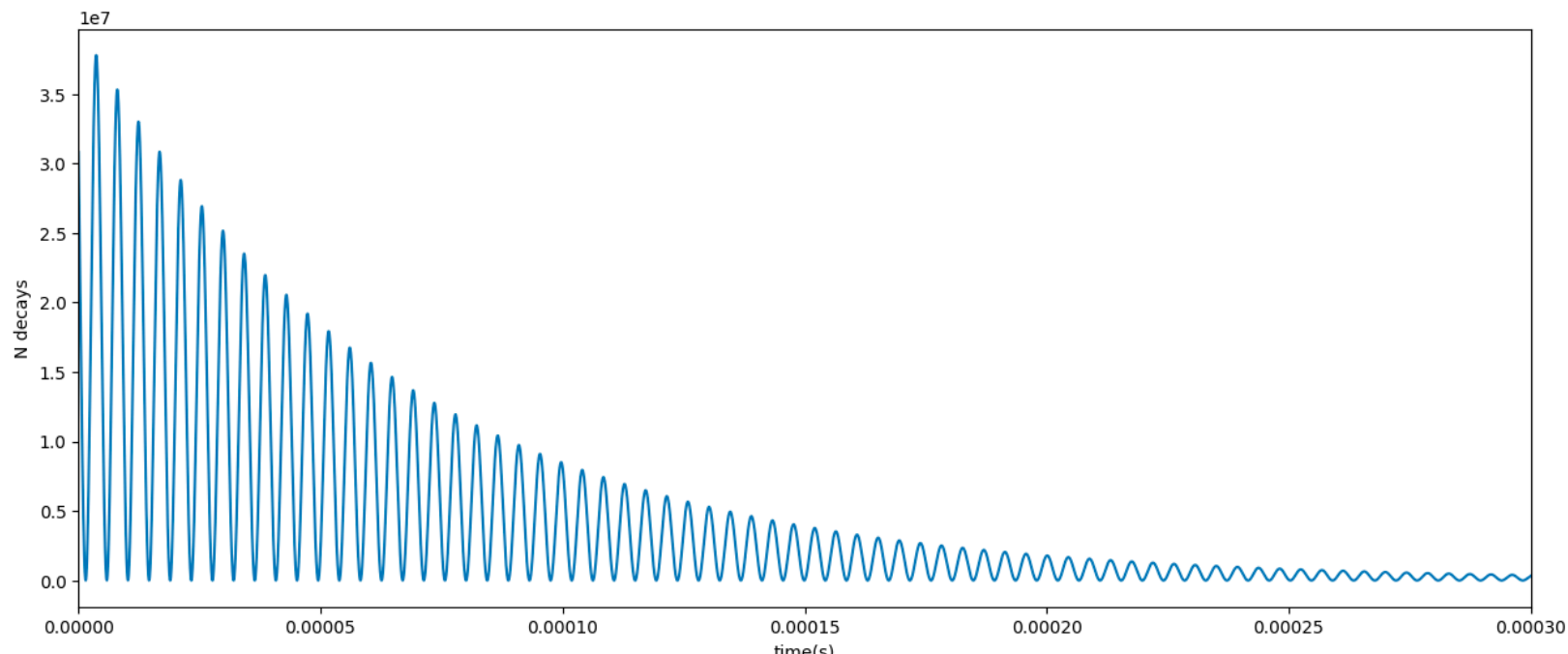


Use a $\chi^2(N_0, \omega, \delta) = \sum \frac{(y(t) - N(t))^2}{\sqrt{y(t)}^2} = \frac{(y(t) - N(t))^2}{y(t)} =$

What is in the denominator? It is the uncertainty on a count N, which from Poisson statistics goes as the sqrt(N)

$$N(t) = N_0 e^{-t/\tau} (1 + \cos(\omega t + \delta))$$

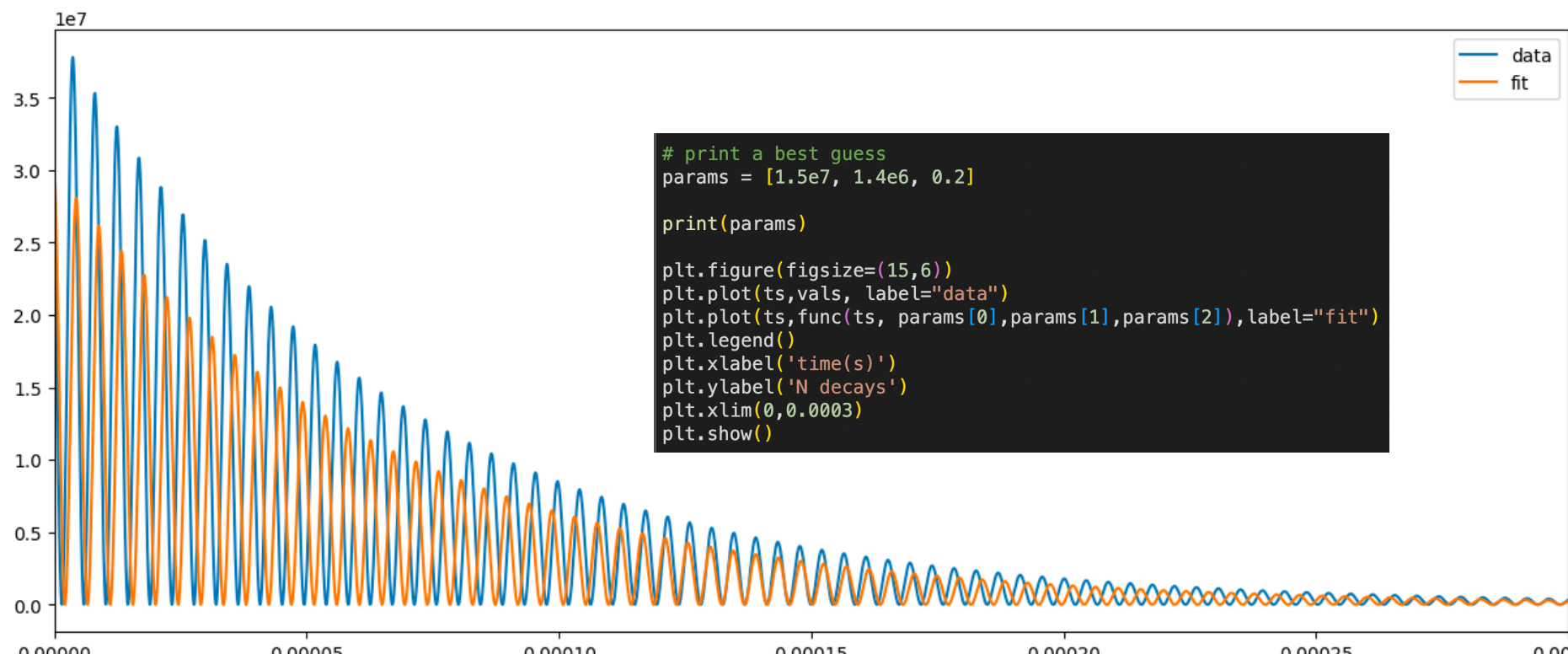
We want to minimize the χ^2 over all 3 parameters! (The lifetime τ is known and fixed)



So how do we move in this space?

Calculate gradient of the χ^2 - and move in the opposite direction! A few tricks: 1) We want to take small steps, 2) We need an initial guess that is close to the true fit values, 3) We want to take smaller and smaller steps as our fit progresses

Clearly not a great fit, but not a bad first guess



Calculating partial derivatives

Use a $\chi^2(N_0, \omega, \delta) = \sum \frac{(y(t) - N(t))^2}{y(t)}$

$$N(t) = N_0 e^{-t/\tau} (1 + \cos(\omega t + \delta))$$

What is the gradient? Calculate partial derivatives!

$$\frac{\partial \chi^2}{\partial N_0} = \sum -2 \frac{(y(t) - N(t))(e^{-t/\tau} (1 + \cos(\omega t + \delta)))}{y(t)}$$

$$\frac{\partial \chi^2}{\partial \omega} = \sum 2 \frac{(y(t) - N(t))(N_0 t e^{-t/\tau} \sin(\omega t + \delta))}{y(t)}$$

$$\frac{\partial \chi^2}{\partial \delta} = \sum 2 \frac{(y(t) - N(t))(N_0 e^{-t/\tau} \sin(\omega t + \delta))}{y(t)}$$

Calculating partial derivatives

Note - we can calculate these analytically! But we can also do this computationally (we'll get there in a bit!)

$$\text{Use a } \chi^2(N_0, \omega, \delta) = \sum \frac{(y(t) - N(t))^2}{y(t)}$$

$$N(t) = N_0 e^{-t/\tau} (1 + \cos(\omega t + \delta))$$

What is the gradient? Calculate partial derivatives!

$$\frac{\partial \chi^2}{\partial N_0} = \sum -2 \frac{(y(t) - N(t))(e^{-t/\tau} (1 + \cos(\omega t + \delta)))}{y(t)}$$

$$\frac{\partial \chi^2}{\partial \omega} = \sum 2 \frac{(y(t) - N(t))(N_0 t e^{-t/\tau} \sin(\omega t + \delta))}{y(t)}$$

$$\frac{\partial \chi^2}{\partial \delta} = \sum 2 \frac{(y(t) - N(t))(N_0 e^{-t/\tau} \sin(\omega t + \delta))}{y(t)}$$

Calculating partial derivatives

```

from numpy import sqrt
tau = 64.4e-6  ← Lorentz-boosed lifetime of muon in lab frame (in s)
def func(t,N0,omega,phase): Our fit function!
    return N0*np.exp(-t/tau)*(1+np.cos(omega*t+phase))

def calcChi2(xs,ys,params,reduced = True): ### params = N0, omega, phase
    chi2 = 0
    for x,y in zip(xs,ys):
        expected = func(x,params[0],params[1],params[2])
        if(y < 1e-6): chi2 += 9 ### placeholder
        else: chi2 += ((y-expected)**2)/y  Need to be careful if denominator gets too small -  $\chi^2$  method breaks down!
    if (reduced): return chi2/len(xs)
    else: return chi2

def gradient(xs,ys,params):
    gradN0 = 0
    gradOmega = 0
    gradDelta = 0
    for x,y in zip(xs,ys):
        denom = y
        if (denom < 1e-6): denom = 9
        gradN0 += -2*(y-func(x,params[0],params[1],params[2]))*(np.exp(-x/tau)*(1+np.cos(params[1]*x+params[2])))/denom
        gradOmega += 2*(y-func(x,params[0],params[1],params[2]))*(params[0]*x*np.exp(-x/tau)*np.sin(params[1]*x+params[2]))/denom
        gradDelta += 2*(y-func(x,params[0],params[1],params[2]))*(params[0]*np.exp(-x/tau)*np.sin(params[1]*x+params[2]))/denom
    return (gradN0,gradOmega,gradDelta)

params = [N0,omega,phase]
chi2 = calcChi2(ts,vals,params)
print(chi2)

```

The challenge with these fits

```

params = [2.0e7, 1.439e6, 1.0]
#params = [2.0e7, 1439337.1142358, 1.0] ### true values

print(params, calcChi2(ts, vals, params))

lr0 = 1e2
lr1 = 1e-4
lr2 = 1e-17
decay = 0.996

chi2s = []
N0s = []
omegas = []
phis = []
gs0 = []
gs1 = []
gs2 = []
for iter in range(1000):
    chi2 = calcChi2(ts, vals, params)
    if (iter%100 == 0): print(iter, chi2, params)
    gradients = gradient(ts, vals, params)
    chi2s.append(chi2)
    N0s.append(params[0])
    omegas.append(params[1])
    phis.append(params[2])
    gs0.append(gradients[0])
    gs1.append(gradients[1])
    gs2.append(gradients[2])

    #params[0] = params[0] - lr0*gradients[0]
    params[1] = params[1] - lr1*gradients[1]
    #params[2] = params[2] - lr2*gradients[2]

```

Initial value, close to true values!

No set rule for these! Small but not too small that we never reach the global minimum

Fits with trig functions are challenging - we really should do a Fourier analysis instead (later chapters)! So let's fit just for ω

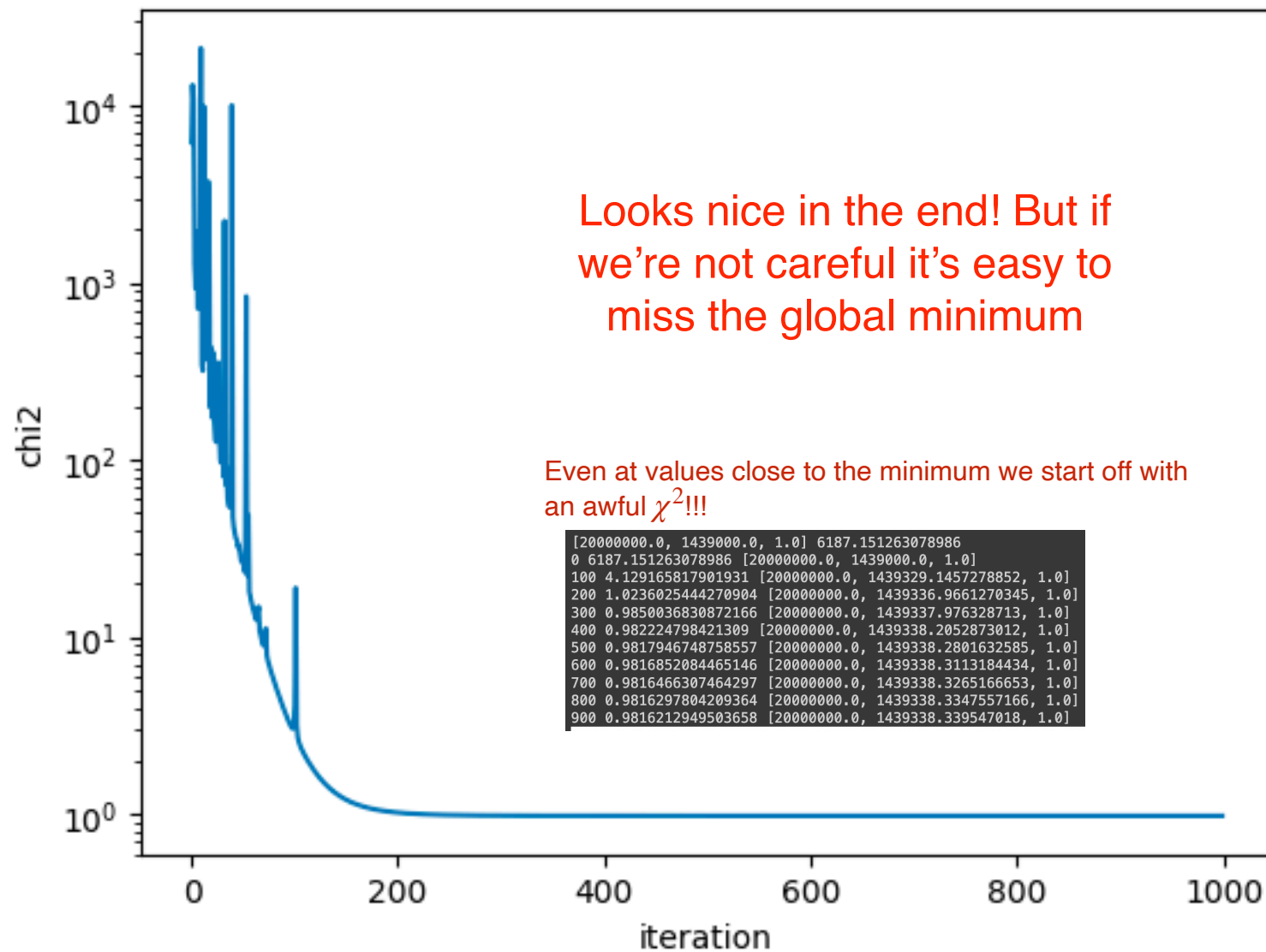
Make our steps smaller as we approach the minimum

```

lr0 = lr0*decay
lr1 = lr1*decay
lr2 = lr2*decay

```

Our fit result



If we normalize our fit function, then for any set of parameters we know the probability that the fit function gave any one data point - it's the function itself! Call that the probability of the data point

Given a set of parameters for our normalized fit function, we can multiply the probabilities for each of our data points to give the joint probability (our likelihood, L) of all data!

Multiplying probabilities is OK, but we end up with very large or small numbers. And it's hard to maximize the joint probability in many cases. To make this easier we can take the log of the likelihood and maximize that instead of the probability with respect to all parameters (the natural log, $\ln$, is a monotonically increasing function, so this still works!)

This is a cool way to answer the question: “For which parameter value does the observed data have the biggest probability?”

Let's give a concrete example

Fit data to function $f(x) = kx^2e^{cx}$, where k is a normalization term. Imagine that c is negative and our data of N points goes from 0 to infinity (in practice we will cut this off at some large value). The constant k is not a free parameter but is really $k(c)$, as we must set k to normalize our fit function

Then $L = \prod_i kx_i^2e^{cx_i}$, and $\ln L = \ln \prod kx_i^2e^{cx_i}$

$\ln L = [\sum_i \ln(k) + \ln(x_i^2) + cx_i] = N \ln k + 2\sum_i \ln(x_i) + c\sum_i x_i$

If we want to maximize $\ln L$ we can also maximize $\ln L/N$

$\ln L/N = \ln k + 2\langle \ln(x) \rangle + c\langle x \rangle$. Looks simple... but what is k ?

We can estimate $k(c)$ numerically for a given c , but thankfully Wikipedia gives us this analytical answer!

$$\int x e^{cx} dx = e^{cx} \left(\frac{cx - 1}{c^2} \right) \quad \text{for } c \neq 0;$$

$$\int x^2 e^{cx} dx = e^{cx} \left(\frac{x^2}{c} - \frac{2x}{c^2} + \frac{2}{c^3} \right)$$

$$\int x^n e^{cx} dx = \frac{1}{c} x^n e^{cx} - \frac{n}{c} \int x^{n-1} e^{cx} dx$$

Let's normalize our fit function

$$\int_0^{\infty} kx^2 e^{cx} = 0 - ke^0 (0/c - 0/c^2 + 2/c^3) = -2k/c^3 = 1,$$

So $k = -c^3/2$ and $f(x) = -\frac{c^3}{2}x^2 e^{cx}$. Then:

$$\ln L/N = \ln (-c^3/2) + 2\langle \ln(x) \rangle + c\langle x \rangle.$$

$$\int x e^{cx} dx = e^{cx} \left(\frac{cx - 1}{c^2} \right) \quad \text{for } c \neq 0;$$

$$\int x^2 e^{cx} dx = e^{cx} \left(\frac{x^2}{c} - \frac{2x}{c^2} + \frac{2}{c^3} \right)$$

$$\int x^n e^{cx} dx = \frac{1}{c} x^n e^{cx} - \frac{n}{c} \int x^{n-1} e^{cx} dx$$

Let's maximize $\ln(L/N)$

$$\int_0^{\infty} kx^2 e^{cx} = 0 - ke^0 (0/c - 0/c^2 + 2/c^3) = -2k/c^3 = 1,$$

So $k = -c^3/2$ and $f(x) = -\frac{c^3}{2}x^2 e^{cx}$. Then:

$\ln L/N = \ln(-c^3/2) + 2\langle \ln(x) \rangle + c\langle x \rangle$. Given a set of data, the only free parameter is c . How to maximize $\ln(L/N)$ for this data? The derivative with respect to c but must be zero!

$$\frac{\partial(\ln L/N)}{\partial c} = 3/c + \langle x \rangle = 0 \rightarrow c = -3/\langle x \rangle, \text{ which is good, it's}$$

negative as we wanted! Is this going to minimize or maximize the probability? Take the second derivative:

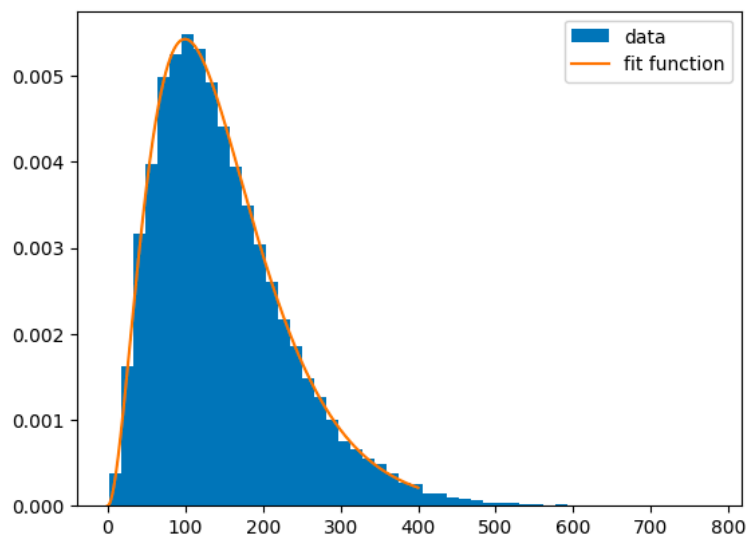
$$\frac{\partial^2(\ln L/N)}{\partial c^2} = -3/c^2, \text{ which is negative, so it's a maximum!}$$

Let's try it out

```
import matplotlib.pyplot as plt
import numpy as np
from google.colab import files
uploaded = files.upload()### first fit data
points = np.genfromtxt('fitdata.txt')
N = points.size
average = np.sum(points)/N
c_estimated = -3/average
print("c estimated = ",c_estimated)
min=0
max=400

def func_fixed(xs):
    return -c_estimated*c_estimated*c_estimated/(2)*xs*xs*np.exp(c_estimated*xs)

plt.hist(points,bins = 50, density = True,label="data")
plt.plot(np.linspace(min,max,1000), np.frompyfunc(func_fixed, 1, 1)(np.linspace(min,max,1000)),label="fit function")
plt.legend()
plt.show()
```



Cool! We didn't even need to do any work. Sometimes we can't get an analytical answer like this but then we can scan across parameters and find the values that maximize the log-likelihood (using some techniques from this and later chapters)

Let's try another one

Fit data to function $f(x) = kx^2 e^{-(ax^2+bx)}$, where k is a normalization term and $a > 0$. Imagine that data of N points goes from -infinity to infinity (in practice we will cut this off at some values). The constant k is not a free parameter but is really $k(a,b)$, as we must set k to normalize our fit function

This is a different way to write a Gaussian centered not at zero!

Then $L = \prod_i k x_i^2 e^{-(ax_i^2+bx_i)}$, and $\ln L = \ln \prod_i k x_i^2 e^{-(ax_i^2+bx_i)}$

$$\begin{aligned} \ln L &= \sum_i \ln(k) + \ln(x_i^2) - ax_i^2 - bx_i = \\ N \ln k + 2N \langle \ln(x) \rangle - aN \langle x^2 \rangle - bN \langle x \rangle \end{aligned}$$

If we want to maximize $\ln L$ we can also maximize $\ln L/N$

$\ln L/N = \ln k + 2\langle \ln(x) \rangle - a\langle x^2 \rangle - b\langle x \rangle$. What is k ?

Let's normalize our second fit function

$$\int_0^{\infty} kx^2 e^{-(ax^2+bx)} = k \sqrt{\frac{\pi}{a}} e^{b^2/(4a)}$$

$$\text{So } k = \sqrt{\frac{a}{\pi}} e^{-b^2/(4a)} \text{ and } f(x) = \sqrt{\frac{a}{\pi}} e^{-b^2/(4a)} x^2 e^{-(ax^2+bx)}$$

We had:

$$\ln L/N = \ln k + 2\langle \ln(x) \rangle - a\langle x^2 \rangle - b\langle x \rangle$$

$$\text{So } \ln L/N = 0.5(\ln a - \ln \pi) + \frac{-b^2}{4a} + 2\langle \ln(x) \rangle - a\langle x^2 \rangle - b\langle x \rangle$$

How do we maximize this? BOTH partial derivatives should be zero!

Let's normalize our second fit function

$$\ln L/N = 0.5(\ln a - \ln \pi) + \frac{-b^2}{4a} + 2\langle \ln(x) \rangle - a\langle x^2 \rangle - b\langle x \rangle$$

$$\frac{\partial \ln L/N}{\partial a} = 0.5/a + \frac{b^2}{4a^2} - \langle x^2 \rangle = 0$$

$$\frac{\partial \ln L/N}{\partial b} = \frac{-b}{2a} - \langle x \rangle = 0$$

Rewrite the first of this: $0.5a + b^2/4 - a^2\langle x^2 \rangle = 0$

From the second: $b = -2a\langle x \rangle$

Plus this back in: $0.5a + a^2\langle x \rangle^2 - a^2\langle x^2 \rangle = 0$. We don't want $a=0$

So $0.5 + a\langle x \rangle^2 - a\langle x^2 \rangle = 0$, or **$a = 1./(2*(\langle x^2 \rangle - \langle x \rangle^2))$**

And then **$b = -\langle x \rangle/(\langle x^2 \rangle - \langle x \rangle^2)$**

Trying out our second fit

```
import matplotlib.pyplot as plt
import numpy as np
from google.colab import files

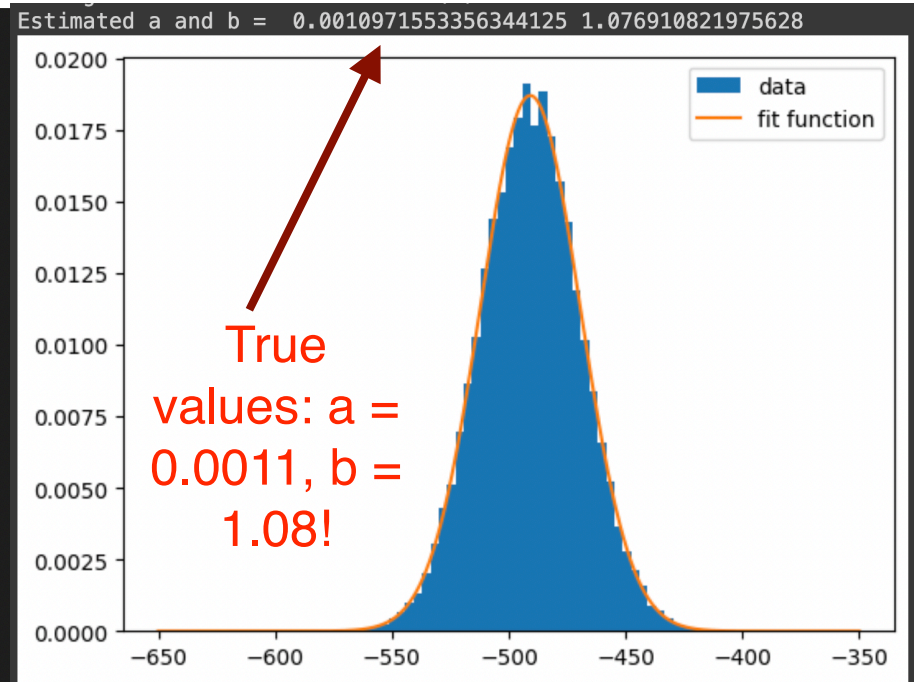
def func_fixed(xs):
    k = np.sqrt(np.pi/a)*np.exp(b*b/(4*a))
    return (1./k)*np.exp(-(a*xs*xs + b*xs))
```

```
uploaded = files.upload()### first fit data
points = np.genfromtxt('fitdata3.txt')
```

```
min=-650
max=-350
```

```
N = points.size
avgx = np.sum(points)/N
avgx2 = np.sum(points*points)/N
a = 1./(2*(avgx2 - avgx*avgx))
b = -avgx/(avgx2 - avgx*avgx)
print("Estimated a and b = ",thisa,thisb)
```

```
plt.hist(points,bins = 50, density = True,label="data")
plt.plot(np.linspace(min,max,1000), np.frompyfunc(func_fixed, 1, 1)(np.linspace(min,max,1000)),label="fit function")
plt.legend()
plt.show()
```



It's great when we can do this analytically! We won't have time to try fits where that isn't possible, for a second semester of this course? :)

<https://classroom.github.com/a/D7PwP7rF>

- 1) Write **pseudocode** to take a set of N (x,y) points, each with associated errors, and return the best fit straight line, errors on parameters, and χ^2/ndf . Include any error checking that might be needed
- 2) Read the data from Hubble's original paper and plot it
- 3) Write actual code to determine the best-fit line and plot that on the same axis, assuming constant errors. How does your value for Hubble's constant (the slope) compare with current best estimates of ~ 70 (km/s)/Mpc? Plot that current best estimate on the same axis. Add legends and axis labels
- 4) Do the same for Problem 3.8 in the textbook. Then redo the problem assuming a constant fractional error of 2% on the y values in the data. Give the χ^2/ndf and errors on a, b and h in this new version. How do your values of a, b and h change? Then redo this again assuming a constant fractional error of 1% on the values in the data added in quadrature with a 0.03V error. How do your χ^2/ndf values change? How do the errors on your parameters change? Providing proper errors on your data is important!

Homework #2 (continued)

- 5) For PHYS510 only: On the course git page you will find a comma-separated file containing several thousand entries for baseball players over the past 5 years. Plot the batting average (the fraction of the time a batter succeeds) as a function of average launch angle (angle at which the ball leaves the baseball bat with respect to the horizontal). Fit this data to a second order polynomial and show your fit on the data. You can use numerical recipes you find on the web or elsewhere for this. What does your 2nd order fit show as the best average angle to succeed (ie to have a hit)? And what does it show as the drop in expected average if a ball is hit with a shift of 10 degrees from optimal?

If you are using python you may want to investigate the csv package. If you want to use colab, you will need to figure out how to get the data into colab. I suggest uploading to your private google drive:

```
from google.colab import drive
drive.mount('/content/drive')
!ls "/content/drive/My Drive/baseball_stats.csv" ## to change if file moves
f = open('/content/drive/My Drive//baseball_stats.csv') # something like this
```